FL 実験コード 利用マニュアル

更新ベクトルレベルの悪性更新検知（研究計画書 v2 対応版）

3CDA2216 笠原秀星

2026 年 9 月 1 日

1 このマニュアルについて

本書は，`sotsuron_plan2.tex`（研究計画書 改訂版）で定義した「Federated Learning における悪性更新の検知可能性」実験のコードの使い方をまとめる。旧版マニュアル（Fang らの Adaptive Model Poisoning Attack 再現実験用）を，検知評価スイープ用に作りなおしたものである。

実験の目的・仮説・評価指標の定義は `sotsuron_plan2.tex` を参照すること。本書はあくまで「どのコマンドで何が動き，どのファイルが出るか」を説明する。

前提とする実験の骨子。

- データ分布の不均一性（Dirichlet の α）と攻撃のステルス性を 2 軸に，毎ラウンドの更新ベクトルから悪性クライアントを見分けられるか（ROC-AUC）を測る。
- 集約手法は検知に非依存なため `fedavg` に固定する（旧実験の Median/Krum 等は使わない）。
- 最終精度ではなく更新ベクトルレベルの検知性能を主指標にするため，学習の収束は必須ではない（詳細は §8）。

2 セットアップ

2.1 必要なもの

- Python 3.10 以上
- 依存パッケージ：`flwr[simulation]`，`flwr-datasets[vision]`，`torch`，`torchvision`，`numpy`，`pandas`，`matplotlib`
- **scikit-learn は不要**。検知器・検知性能指標はすべて `detection.py` 内に numpy だけで実装してある。
- GPU は不要（CPU のみで完結する規模に設定してある）。

2.2 インストール

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
```

`pyproject.toml` の `dependencies` に `flwr` などが定義されている。`pandas` / `matplotlib` が未導入なら追加で `pip install pandas matplotlib`。

2.3 初回実行時の注意（MNIST ダウンロード）

初回の `flwr run` で HuggingFace Hub から MNIST（約 11 MB）が自動ダウンロードされる。`Warning: You are sending unauthenticated requests to the HF Hub` が出ても動作に支障はない。レート制限が気になる場合のみ `export HF_TOKEN=...` を設定する。

3 ファイル構成

表 1　本実験に関係するファイル（★＝今回の検知実験で新規／改修）

ファイル	役割
detection.py ★	検知器 D1–D4 と検知性能指標（ROC-AUC / AP / recall@f / precision@f）。純 numpy。
attack.py ★	攻撃の実装。今回 alie_attack（ALIE ステルス攻撃）を追加。
aggregation.py	集約手法（本実験では fedavg のみ使用）。
pytorchexample/task.py	モデル（CNN）とデータ分割（iid / non_iid / dirichlet）。
pytorchexample/client_app.py	クライアント側の学習。sign_flip 攻撃はここで適用。
pytorchexample/server_app.py ★	サーバ側の全体制御。alie-z 等の設定読み込みを追加。
pytorchexample/robust_strategy.py ★	集約直前フック。ALIE 攻撃の適用と，更新ベクトルへの D1–D4 スコア計算・記録。
pytorchexample/log_output.py ★	log_detection_csv（1 クライアント 1 行の検知ログ）を追加。
run_experiments.py ★	スイープ実行。--detection を追加。
analyze_detection.py ★	検知ログの後処理（指標 CSV・図の生成）。
tools/verify_dirichlet_partition.py ★	Dirichlet 分割の目視確認スクリプト。
test_detection.py ★	detection.py の単体テスト。
test_attack.py / test_aggregation.py / test_malicious.py	既存の単体テスト。

4 設定パラメータ

すべて pyproject.toml の [tool.flwr.app.config] に既定値があり，flwr run の --run-config で 1 回だけ上書きできる。本実験に関係するものを表 2 に示す。

表 2　本実験で使う run-config キー

キー	本実験での値	意味
data-split	"dirichlet"	データ分割方式。iid / non_iid（Fang 方式）/ dirichlet。
dirichlet-alpha	100 / 0.5 / 0.1	Dirichlet の濃度。大きいほど IID 寄り，小さいほど不均一。
attack-type	"none" / "sign_flip" / "alie"	攻撃。none ＝攻撃なし（基準），sign_flip ＝粗い攻撃（A1，クライアント側で Δ 反転），alie ＝ステルス攻撃（A2，サーバ側で $\mu - z\sigma$）。
alie-z	1.0	ALIE の摂動強度 z（正常の標準偏差の何倍ずらすか。実用上 0.5–1.5）。
malicious-ratio	0.2（攻撃なしは 0.0）	悪性クライアント割合。
num-clients	20	クライアント総数。--federation-config num-supernodes と一致させる。
num-server-rounds	30	通信ラウンド数。
aggregation-method	"fedavg"	集約手法（本実験は固定）。
trial	1 / 2 / 3	繰り返し試行番号。データ分割・モデル初期値・悪性選出のすべてに連動。
malicious-seed	trial と同じ値	悪性クライアント選出のシード。run_experiments.py は trial と同値を渡す。
output-dir	条件ごとの出力先	final_model.pt の保存先（CSV ログは常に logs/ に出る）。

5　実行手順

5.1　手順 0：単体テスト

コードに触れたら，まず検知器のテストを回す。

```
python3 test_detection.py
python3 test_aggregation.py
python3 test_malicious.py
```

test_attack.py は test_adaptive_attack_noop_for_fedavg が失敗するが，これは本実験の前からある既知の失敗（旧 Fang 実験の古いテストと現行実装の不整合）であり，今回の検知実験には影響しない。ALIE の追加分だけ確認したい場合は次のようにする。

```
python3 -c "import test_attack as T;

  [getattr(T,n)() for n in dir(T) if n.startswith('test_alie')]"
```

5.2　手順 1：Dirichlet 分割の目視確認（ゲート G1 の前）

各 α で，クライアントごとの保有クラスがどれだけ偏るかを確認する。

```
python3 tools/verify_dirichlet_partition.py --alpha 100 0.5 0.1

  --clients 20 --trial 1
```

- 標準出力：クライアント × クラスの件数表，1 クライアントあたりのサンプル数と保有クラス数の要約。

- 画像：results_detection/dirichlet_partition_a{alpha}_trial1.png（保有割合のヒートマップ）。
- 期待：$\alpha = 100$ はほぼ均一，$\alpha = 0.1$ では一部クライアントの保有クラスが数個まで減る。

5.3 手順 2：動作確認（小規模の単発実行）

スイープ前に，1 条件だけ短く回してログが出るか確認する。

```
flwr run . local-simulation --stream

  --federation-config num-supernodes=6

  --run-config 'num-server-rounds=2 aggregation-method="fedavg"

    data-split="dirichlet" dirichlet-alpha=0.5

    attack-type="alie" alie-z=1.0 malicious-ratio=0.2

    malicious-seed=1 trial=1 num-clients=6 output-dir="_smoke"'
```

完了後，logs/detection_updates.csv が生成され，1 ラウンドあたり num-clients 行（is_malicious 列に 0/1）が追記されていることを確認する。確認用の出力（_smoke/，logs/）は消してよい。

5.4 手順 3：検知評価スイープ

```
python3 run_experiments.py --detection
```

- 条件：$\alpha \in \{100, 0.5, 0.1\}$ × 攻撃 ∈ {none, sign_flip, alie} × trial ∈ $\{1, 2, 3\}$ = **27 条件**。fedavg 固定，悪性率 0.2（none のみ 0.0），20 クライアント，30 ラウンド。
- 出力先：experiments_detection/<条件名>/final_model.pt。このファイルがあれば「完了済み」としてスキップするので，中断→再実行で未完了分だけ再開できる。
- 失敗した条件は experiments_detection/failures.csv に記録され，スイープは止まらない。
- CSV ログ（logs/detection_updates.csv など）は全条件が追記していく。
- 目安：1 条件あたり CPU で数分，全体で 1–2 時間程度。

条件を絞りたい場合の例（run_experiments.py 冒頭の DETECTION_* 定数を編集する）：DETECTION_ALPHAS，DETECTION_ATTACKS，DETECTION_TRIALS，DETECTION_NUM_ROUNDS など。

5.5 手順 4：後処理と作図

```
python3 analyze_detection.py
```

logs/detection_updates.csv を読み，条件 × ラウンド × 検知器ごとに ROC-AUC などを計算して，以下を出力する。

- logs/detection_metrics.csv
- results_detection/detection_auc_by_round.png：検知器ごとの「AUC 対 ラウンド」（α・攻撃別，trial 平均 ± 標準偏差）。
- results_detection/detection_pushratio_vs_auc.png：**横軸** push_ratio **・縦軸 AUC の散布図**（中心仮説の主図）。
- results_detection/detection_final_auc_table.csv：最終数ラウンドの平均 AUC 一覧（α× 攻撃 × 検知器）。

オプション：--updates（入力 CSV），--outdir（画像出力先），--last-k（最終平均を取るラウンド数，既定 5）。

6 出力ファイルの内容

6.1 logs/detection_updates.csv（1 クライアント 1 行）

集約直前（攻撃適用後・集約前）の各クライアント更新について記録する。検知の入力は更新ベクトル $g_i = w_i - w_{\text{global}}$（受信モデルからグローバルモデルを引いたもの）。

表 3 detection_updates.csv の列

列	内容
run_id	実行を一意に識別する文字列。
round	通信ラウンド番号。
aggregation_method, data_split, alpha, attack_type, malicious_ratio, trial	実験条件。
client_pos	集約時の配列内位置（$0..n-1$）。
partition_id	実際のクライアント ID。
is_malicious	正解ラベル（1 ＝悪性）。**評価専用**。検知器スコアの計算には使わない。
delta_norm	$\|g_i\|_2$。
d1_norm	検知器 D1（ノルム偏差）のスコア。大きいほど悪性らしい。
d2_cosine	検知器 D2（座標中央値へのコサイン距離）。
d3_kmeans	検知器 D3（標準化特徴量への 2-means，正常クラスタ重心までの距離）。
d4_spectral	検知器 D4（第 1 右特異ベクトルへの射影の 2 乗）。
benign_spread	そのラウンドの正常クラスタの広がり（全行で同じ値）。
push_ratio	そのラウンドの push / spread（全行で同じ値。悪性 0 人の行は空）。

6.2 logs/detection_metrics.csv

analyze_detection.py が生成する。1 行＝（条件，ラウンド，検知器）。主な列：auc，ap，recall_at_f，precision_at_f，benign_spread，push_ratio，n_malicious。ここで f は各条件の実際の悪性数。攻撃なし（attack_type="none"）の行は悪性が 0 人なので auc は NaN になる（benign_spread の基準値としてのみ使う）。

6.3 その他のログ（既存）

- logs/diagnostics.csv：ラウンドごとの benign_spread / push_ratio / frac_selected_malicious（旧実験からのメカニズム分析用）。
- logs/results.csv / logs/summary.csv：ラウンドごと／最終の Accuracy・Loss。本実験の主指標ではないが，収束確認（§8）に使う。
- experiments_detection/<条件名>/final_model.pt：最終グローバルモデル。

7 検知器と攻撃の対応

7.1 検知器（detection.py）

記号	スコア	想定される得手・不得手
D1 d1_norm	$\lvert \lVert g_i \rVert - \mathrm{median}_j \lVert g_j \rVert \rvert$	ノルムを変える攻撃に強い。ステルス攻撃に弱い。
D2 d2_cosine	$1 - \cos(g_i, r)$，r = 座標中央値	向きの異常に強い。正常に共通方向が無いと効かない。
D3 d3_kmeans	標準化 [norm, D2, PC1, PC2] への 2-means，正常重心までの距離	総合的。クラスタが分離しないと退化。
D4 d4_spectral	$(g_i \cdot v)^2$，v = 第 1 右特異ベクトル	悪性が同一ベクトルだと強い。min-max 型攻撃はこれを回避する設計。

7.2 攻撃（attack.py / robust_strategy.py）

- sign_flip（A1）：クライアント側で更新量 $\Delta = \text{local} - \text{global}$ の符号を反転して送る粗い攻撃。
- alie（A2）：サーバ側で正常更新の座標ごと平均 μ・標準偏差 σ から $g^{\text{mal}} = \mu - z\sigma$ を作り，悪性クライアント全員に同じ値を割り当てる。集約手法非依存のステルス攻撃（Baruch ら, “A Little Is Enough”, 2019）。
- （参考）adaptive：旧 Fang 実験の集約手法特化攻撃。本実験では使わない。

8 ゲート G1：収束を必要としないことの確認

検知は「そのラウンドの更新ベクトル集合が分離可能か」の問題であり，モデルが最終精度に達している必要はない。ただし，検知性能（AUC）がラウンドをまたいで極端にドリフトすると解釈が難しくなる。

results_detection/detection_auc_by_round.png を見て，AUC の推移が解釈可能な範囲に収まっているかを確認する。大きく暴れる場合は，学習率を上げて収束を早める。

```
# 例：学習率を 0.01 に上げて 40 ラウンドにする
python3 run_experiments.py --detection      # の前に run_experiments.py の
                                            # DETECTION_NUM_ROUNDS と，
# pyproject.toml の learning-rate 既定値，または --run-config で learning-rate=0.01 を渡す
```

9 トラブルシューティング

症状	対処
flwr run が一見成功するのにログが出ない	flwr run は内部エラーでも終了コード 0 を返すことがある。--stream を付け，experiments_detection/<条件名>/final_model.pt の有無で成否を判断する（run_experiments.py はこの方式）。
num-supernodes と num-clients の不一致	--federation-config num-supernodes=N と run-config の num-clients=N を必ず一致させる。
MNIST ダウンロードの警告	HF_TOKEN 未設定の警告は無視してよい。完全にオフラインなら事前に datasets でキャッシュしておく。
検知ログの記録に失敗した旨のメッセージ	検知スコア計算で例外が出ても学習自体は止めない設計。標準出力に理由が出るので確認する。
detection_metrics.csv の auc が NaN	その条件が attack_type="none"（悪性 0 人）なら正常。攻撃ありで NaN なら悪性ラベルが付いていない可能性（malicious-ratio と num-clients を確認）。
メモリ不足	num-clients を下げる。更新ベクトルは（クライアント数 × モデル次元）の行列としてサーバ上に載る。

10 再現性について

trial 番号は，データ分割・モデル初期値・悪性クライアント選出のすべてに連動する。同じ trial なら常に同じ設定が再現され，trial を変えると悪性として選ばれるクライアント ID 自体も変わる。run_experiments.py --detection は malicious-seed に trial と同じ値を渡している。